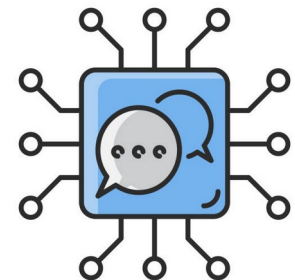
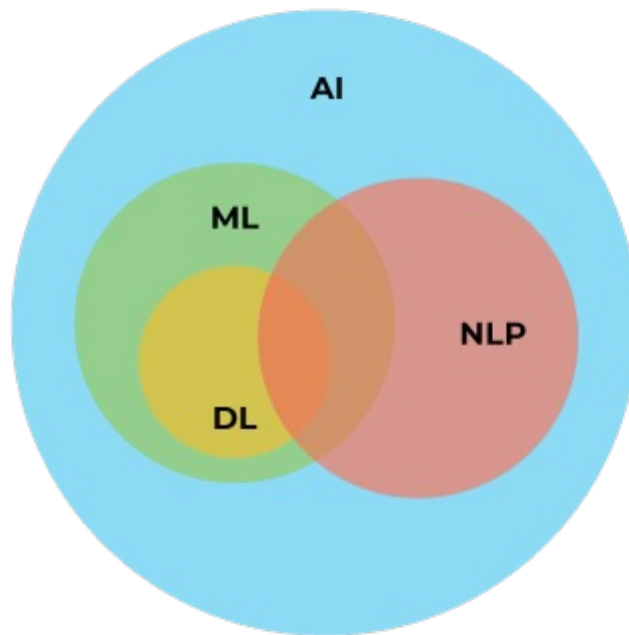


Machine Learning and Deep Learning for NLP

Tushar B. Kute,
<http://tusharkute.com>



Why ML & DL for NLP?



- Artificial Intelligence
- Machine Learning
- Language Processing
- Deep Learning

Why ML & DL for NLP?



Machine learning and Deep Learning for NLP helps data analysts turn unstructured text into usable data and insights.

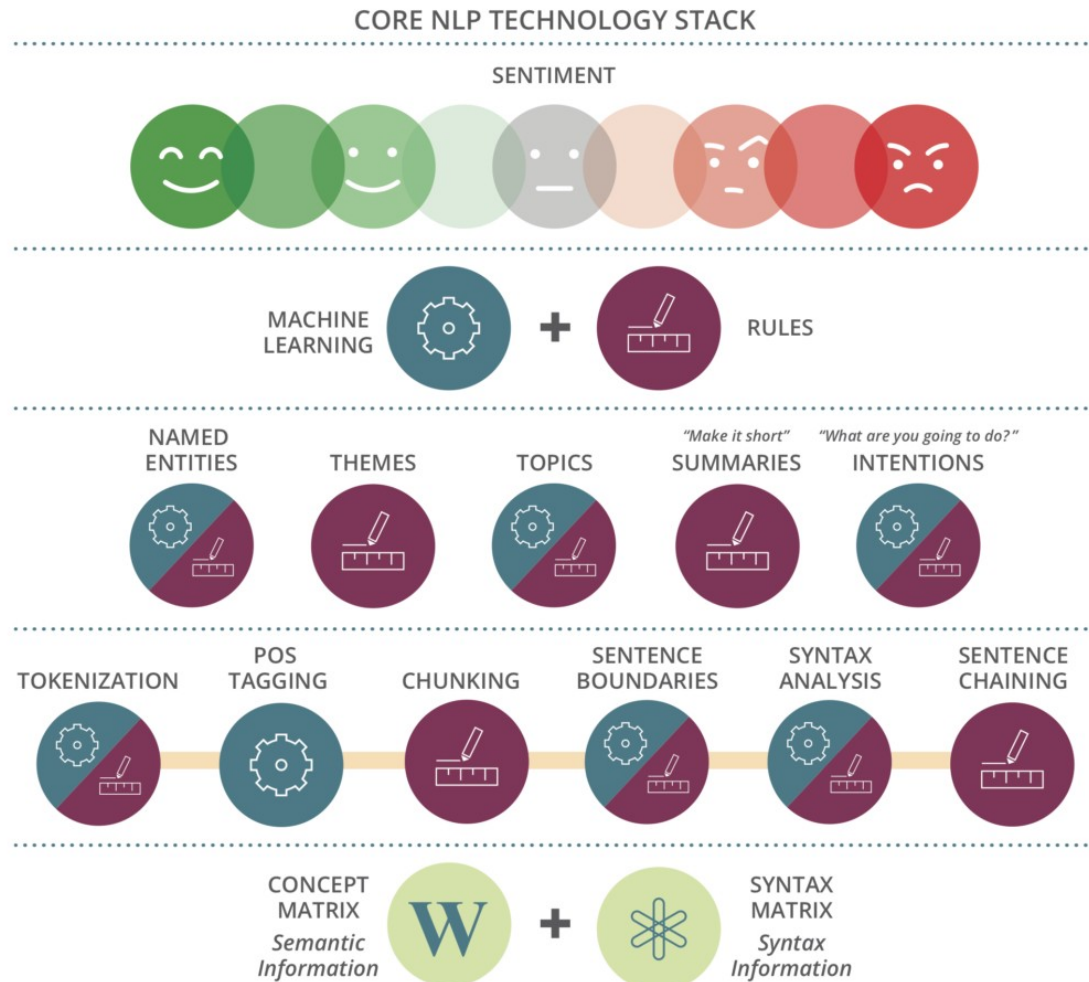
Classification

- Categorization means sorting content into buckets to get a quick, high-level overview of what's in the data.
- To train a text classification model, data scientists use pre-sorted content and gently shepherd their model until it's reached the desired level of accuracy.
- The result is accurate, reliable categorization of text documents that takes far less time and energy than human analysis.

Unsupervised Learning

- Clustering means grouping similar documents together into groups or sets.
- These clusters are then sorted based on importance and relevancy (hierarchical clustering).
- Another type of unsupervised learning is Latent Semantic Indexing (LSI). This technique identifies on words and phrases that frequently occur with each other.
- Data scientists use LSI for faceted searches, or for returning search results that aren't the exact search term.

NLP Technology Stack



Deep Learning vs. Machine Learning

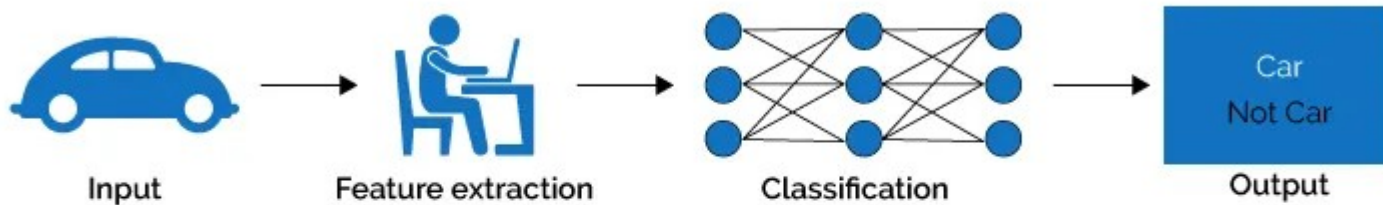
- Deep Learning can essentially do everything that machine learning does, but not the other way around.
- For instance, machine learning is useful when the dataset is small and well-curated, which means that the data is carefully preprocessed.
- Data preprocessing requires human intervention. It also means that when the dataset is large and complex, machine learning algorithms will fail to extract information, and it will underfit.

Deep Learning vs. Machine Learning

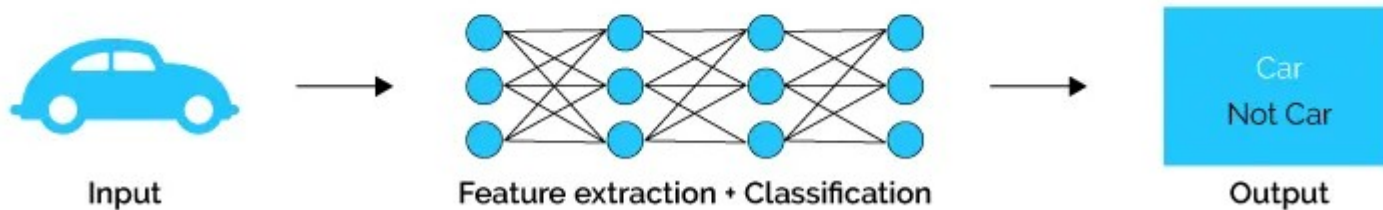
- Generally, machine learning is alternatively termed shallow learning because it is very effective for smaller datasets.
- Deep learning, on the other hand, is extremely powerful when the dataset is large.
- It can learn any complex patterns from the data and can draw accurate conclusions on its own. In fact, deep learning is so powerful that it can even process unstructured data - data that is not adequately arranged like text corpus, social media activity, etc.
- Furthermore, it can also generate new data samples and find anomalies that machine learning algorithms and human eyes can miss.

Deep Learning vs. Machine Learning

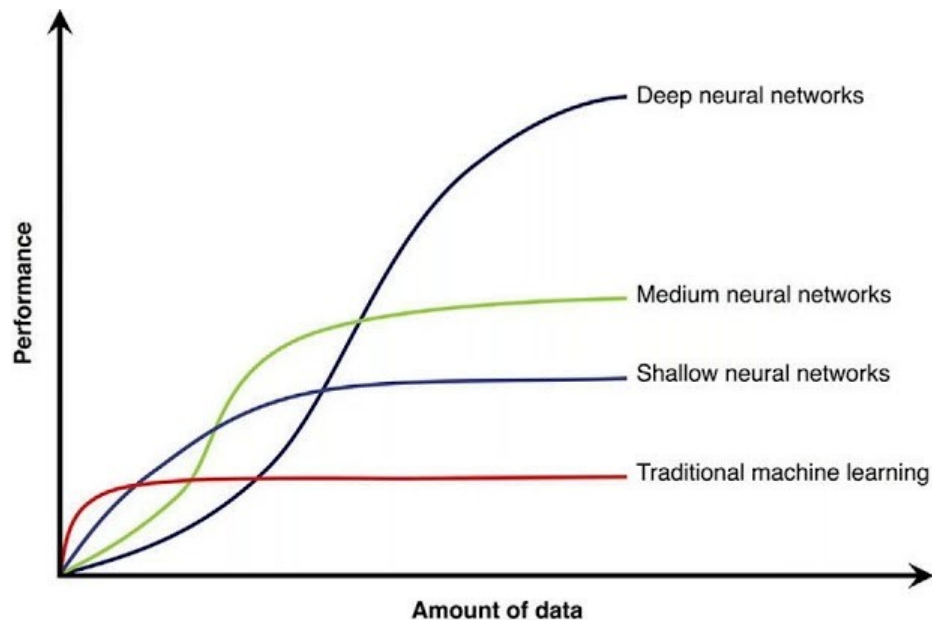
Machine Learning



Deep Learning



Why Deep Learning ?



Why Now?

- *Algorithm Advancements*
- *GPU Computing*
- *Availability of Larger Training Data*

Deep Learning vs. Machine Learning

- On the downside, deep learning is computationally expensive compared to machine learning, which also means that it requires a lot of time to process.
- Deep Learning and Machine Learning are both capable of different types of learning: Supervised Learning (labeled data), Unsupervised Learning (unlabeled data), and Reinforcement Learning.
- But their usefulness is usually determined by the size and complexity of the data.

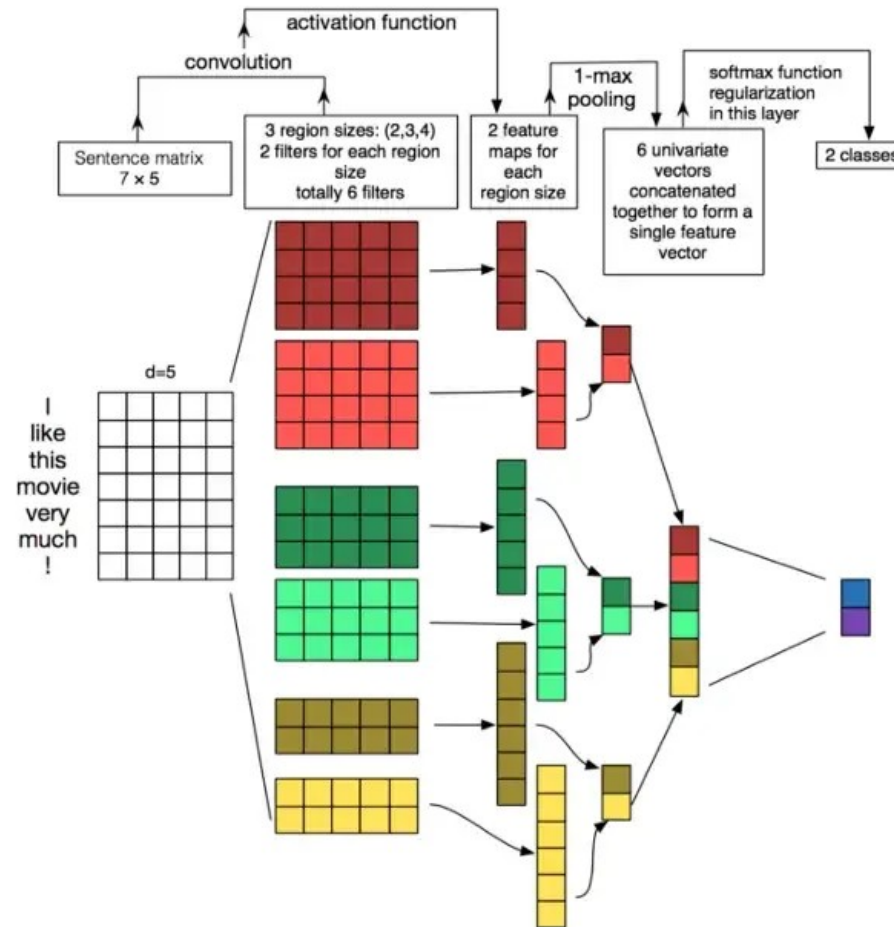
A Quick Summary

- Machine learning requires data preprocessing, which involves human intervention.
- The neural networks in deep learning are capable of extracting features; hence no human intervention is required.
- Deep Learning can process unstructured data.
- Deep Learning is usually based on representative learning i.e., finding and extracting vital information or patterns that represent the entire dataset.
- Deep learning is computationally expensive and time-consuming.

ANN and DNN

- Practical

CNN



Word Embedding

- Word embeddings are a type of word representation that allows words with similar meaning to have a similar representation.
- They are a distributed representation for text that is perhaps one of the key breakthroughs for the impressive performance of deep learning methods on challenging natural language processing problems.

Word Embedding

- Word embeddings are in fact a class of techniques where individual words are represented as real-valued vectors in a predefined vector space.
- Each word is mapped to one vector and the vector values are learned in a way that resembles a neural network, and hence the technique is often lumped into the field of deep learning. Key to the approach is the idea of using a dense distributed representation for each word.
- Each word is represented by a real-valued vector, often tens or hundreds of dimensions. This is contrasted to the thousands or millions of dimensions required for sparse word representations, such as a one hot encoding

Why Do We Need an Embedding Layer?

- The Problem: Neural Networks are mathematical engines. They operate on continuous numbers (additions, multiplications), not text strings.
- The Constraint: A computer cannot calculate "Apple" - "Orange".
- The Goal: We must convert discrete words (vocabulary) into a numerical format that the CNN can process.

One-Hot Encoding vs. Embeddings

- One-Hot Encoding (The Old Way):
 - Represents words as massive vectors of zeros with a single '1'.
 - Sparse: Huge memory waste.
 - No Meaning: "King" and "Queen" are mathematically unrelated.
- Embeddings (The New Way):
 - Represents words as dense lists of decimal numbers.
 - Efficient: Small, fixed size (e.g., 300 dimensions).
 - Trainable: The network learns the meaning; similar words get similar numbers.

Building the Lookup Table

- Vocabulary Size (V): The total count of unique words in our dictionary.
 - Example: 5 words {"I": 0, "love": 1, "AI": 2, "hate": 3, "bugs": 4}.
- Embedding Dimension (D): How many features (numbers) represent one word.
 - Example: $D=3$.
- The Matrix: This creates a 5×3 matrix of weights. Initially random, but refined during training.

Processing the Input

- Input Sentence: "I love AI"
- Integer Conversion: The text is converted to indices based on the vocabulary.
 - [0, 1, 2]
- The Action: The layer performs a "Lookup" (not a matrix multiplication).
 - For Index 0 ("I") → Fetch Row 0
 - For Index 1 ("love") → Fetch Row 1
 - For Index 2 ("AI") → Fetch Row 2

Creating the Feature Matrix

- The layer stacks the fetched vectors to form a 2D matrix.
 - Shape: (Sentence Length × Embedding Dimension)
- Visualizing the Output:

$$\begin{bmatrix} 0.1 & 0.2 & 0.9 \\ \text{(Vector for "I")} & & \\ 0.8 & 0.1 & 0.1 \\ \text{(Vector for "love")} & & \\ 0.2 & 0.8 & 0.3 \\ \text{(Vector for "AI")} & & \end{bmatrix}$$

Feeding the 1D Convolution

- The Result: Our text is now a numerical grid, looking mathematically similar to an "image".
 - CNN Input:
 - Image CNN: Input is (Height $\times$ Width $\times$ Channels).
 - NLP CNN: Input is (Sentence Length $\times$ Embedding Dimension).
 - 1D Convolution: The filter will now slide down the "Sentence Length" axis, analyzing words in groups (bigrams/trigrams) to detect patterns like sentiment or entities.

What is 1D Convolution?

- Definition: A mathematical operation used to extract features from sequential data (like text or audio) by sliding a smaller "window" over the input.
- The "Window": This window is called a Kernel or Filter.
- The Goal: To detect local patterns (like specific phrases in text or spikes in a signal) regardless of their position in the sequence.

Anatomy of a Convolution

- Input (I): The sequence of data to be processed (e.g., word embeddings).
 - Example: [1, 2, 3, 4, 5]
- Kernel (K): A learned set of weights that looks for patterns.
 - Example: [1, 0, -1] (Length = 3)
- Output (O): The resulting map of detected features.

The "Sliding Window" Begins

- Alignment: Place the Kernel at the start of the Input.
- Visual:
 - Input: [1, 2, 3], 4, 5
 - Kernel: [1, 0, -1]
- Operation: Element-wise multiplication followed by a sum (Dot Product).
 - $(1 \times 1) + (2 \times 0) + (3 \times -1)$
 - $1 + 0 - 3 = -2$
- Result: The first number in our output is -2.

Striding Forward

- Action: Slide the Kernel one step to the right.
- Visual:
 - Input: 1, [2, 3, 4], 5
 - Kernel: [1, 0, -1]
- Calculation:
 - $(2 \times 1) + (3 \times 0) + (4 \times -1)$
 - $2 + 0 - 4 = -2$
- Result: The second number in our output is -2.

Completing the Pass

- Action: Slide the Kernel one step further.
- Visual:
 - Input: 1, 2, [3, 4, 5]
 - Kernel: [1, 0, -1]
- Calculation:
 - $(3 \times 1) + (4 \times 0) + (5 \times -1)$
 - $3 + 0 - 5 = -2$
- Result: The third number in our output is -2.

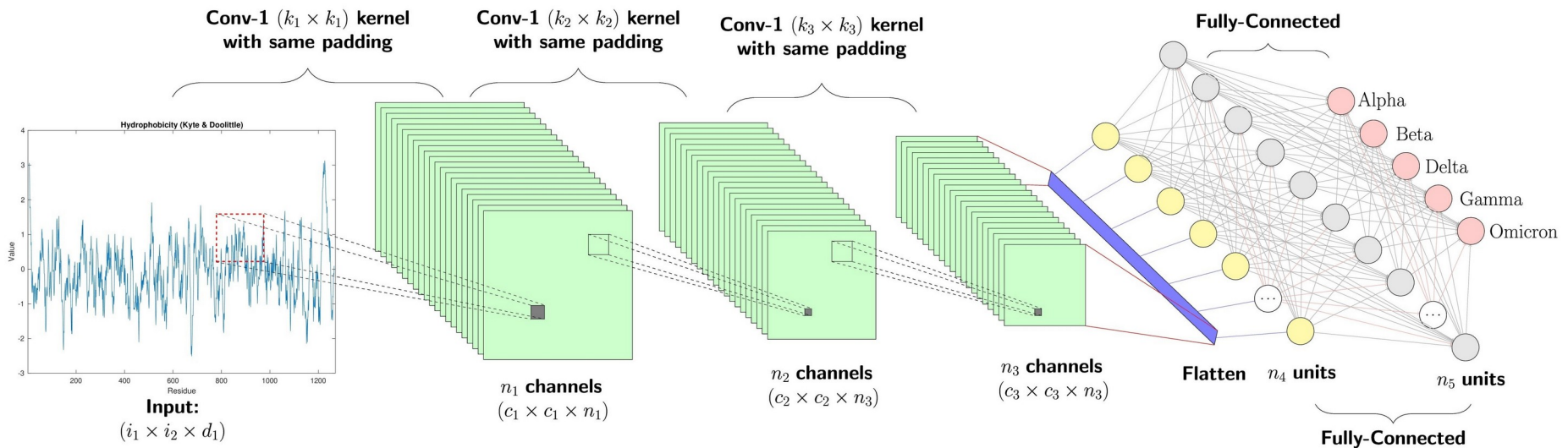
Feature Map Generation

- Input Sequence: [1, 2, 3, 4, 5] (Length 5)
- Kernel: [1, 0, -1] (Length 3)
- Output Sequence: [-2, -2, -2] (Length 3)
- Observation: The output is shorter than the input (Input Length - Kernel Length + 1).
 - *Note:* We can use Padding (adding zeros) if we want to keep the same length.

Why Use This for Language?

- N-Grams: A Kernel of size 2 acts like a Bigram detector; size 3 acts like a Trigram detector.
- Pattern Recognition:
 - If the Kernel represents the pattern for "not good", it will produce a high activation signal whenever it slides over that phrase in a sentence.
- Position Invariance: The CNN will find "not good" whether it appears at the start, middle, or end of the tweet.

Complete Convolution

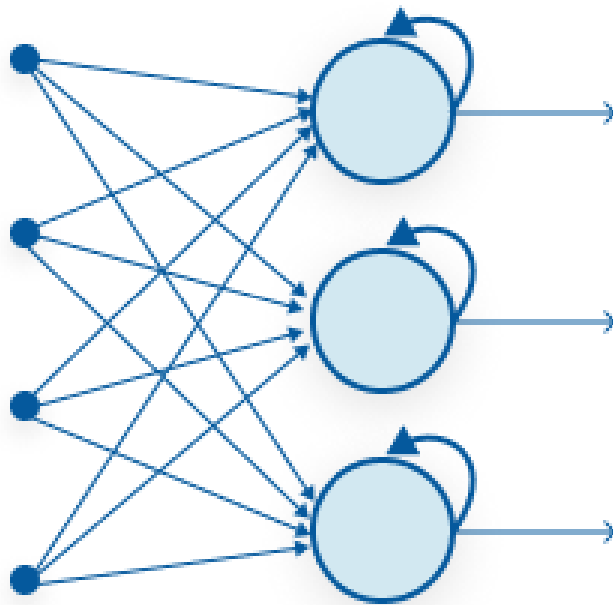


- Practical

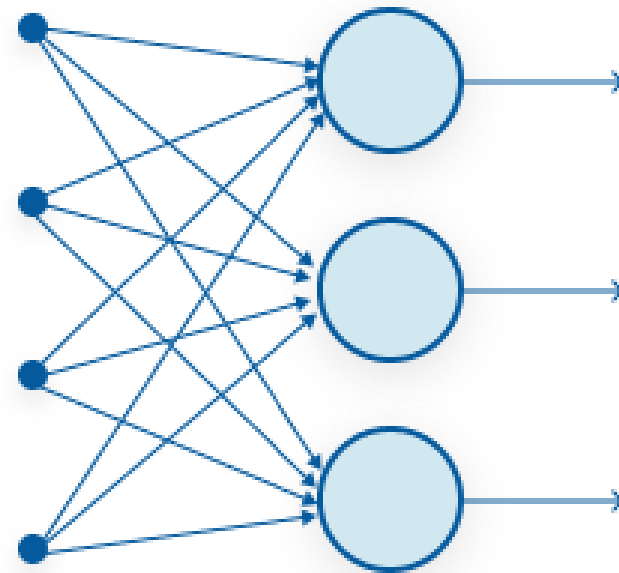
RNN

- A recurrent neural network (RNN) is a class of artificial neural networks where connections between nodes can create a cycle, allowing output from some nodes to affect subsequent input to the same nodes.
- This allows it to exhibit temporal dynamic behavior. Derived from feedforward neural networks, RNNs can use their internal state (memory) to process variable length sequences of inputs.
- This makes them applicable to tasks such as unsegmented, connected handwriting recognition or speech recognition.

RNN

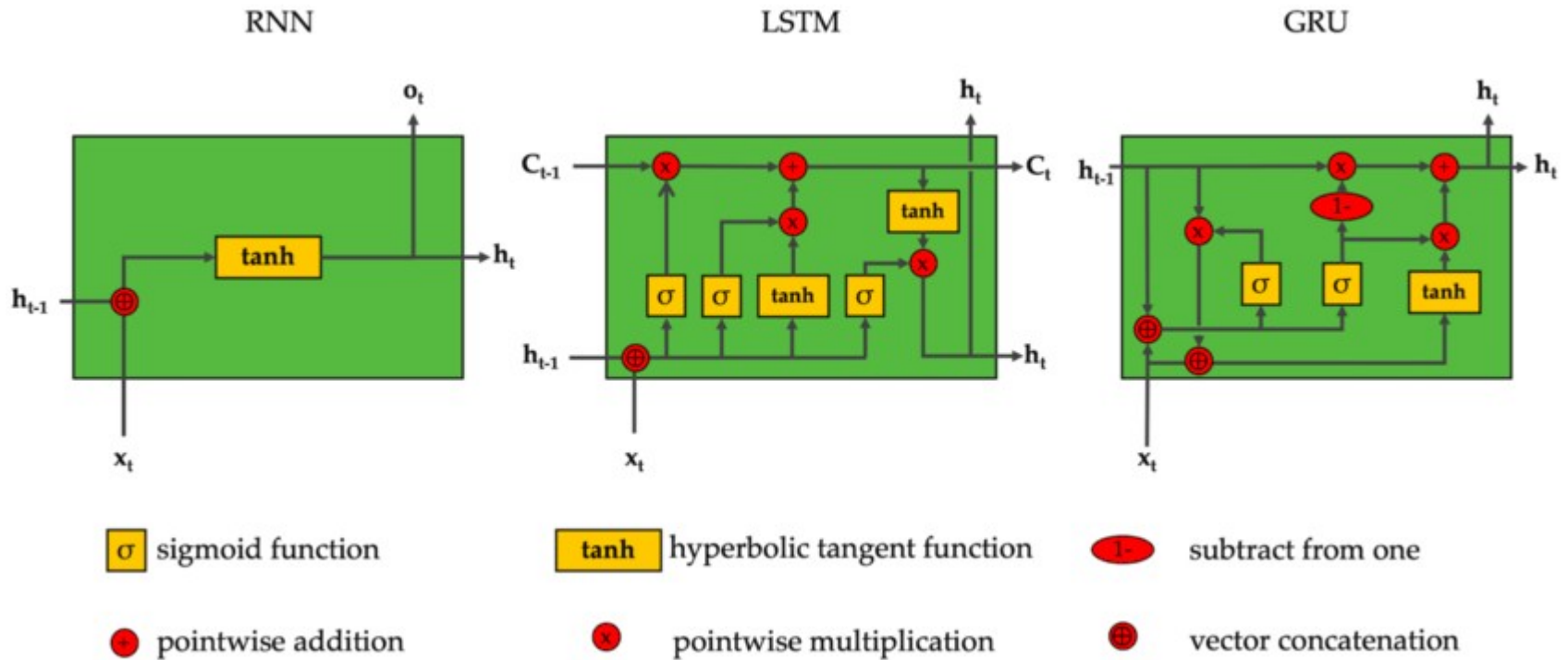


Recurrent Neural Network



Feed-Forward Neural Network

RNN



RNN

- Practical

Thank you

This presentation is created using LibreOffice Impress 7.4.1.2, can be used freely as per GNU General Public License



@mitu_skillologies



@mITuSkillologies



@mitu_group



@mitu-skillologies



@MITUSkillologies

kaggle

@mituskillologies

Web Resources

<https://mitu.co.in>

<http://tusharkute.com>



@mituskillologies

contact@mitu.co.in
tushar@tusharkute.com